

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 1 Duración: 90 minutos

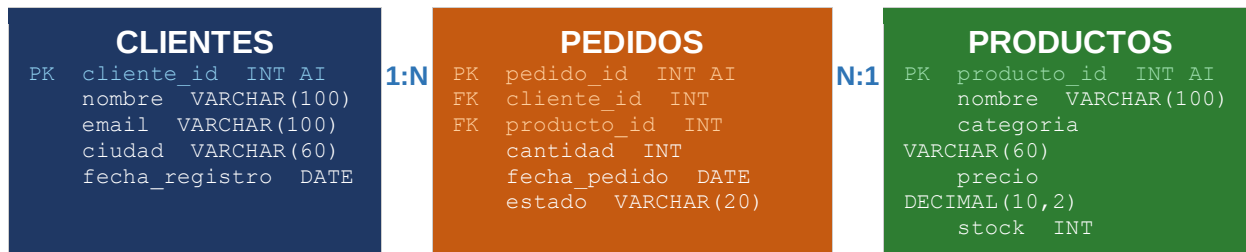
Nombre:		Código:	
Docente:		Fecha:	

INSTRUCCIONES:

1. Cada pregunta integra múltiples temáticas SQL; lea con atención el enunciado completo.
2. Escriba sentencias con sintaxis válida para MySQL
3. Cada estudiante solo debe resolver la pregunta asignada.
4. Se evalúa: correctitud lógica, sintaxis, eficiencia y uso de las cláusulas indicadas.
5. No se permite material de apoyo, celulares ni calculadoras.
6. **Deben entregar:**
 - Pantallazos de los resultados de ejecución
 - Script con las sentencias recuerda documentar el código.
 - Cargar el script y los pantallazos en el repositorio de trabajos en un paquete que se llame Segundo parcial Práctico.
 - Incluir en la plataforma el enlace del repositorio, el Script y el PDF de los pantallazos obtenidos de la ejecución.

MODELO RELACIONAL Y SCRIPT DE BASE DE DATOS

Base de datos: sistema de gestión de tienda de tecnología. Use estas tablas en todas las preguntas.



SCRIPT DDL / DML

```
-- =====
-- DDL: CREACION DE BASE DE DATOS Y TABLAS
-- =====

DROP DATABASE IF EXISTS tienda_tech;
CREATE DATABASE tienda_tech CHARACTER SET utf8mb4;
USE tienda_tech;

CREATE TABLE clientes (
    cliente_id      INT AUTO_INCREMENT PRIMARY KEY,
    nombre          VARCHAR(100) NOT NULL,
    email           VARCHAR(100) UNIQUE NOT NULL,
    ciudad          VARCHAR(60),
    fecha_registro  DATE DEFAULT (CURRENT_DATE)
);
```

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 1 Duración: 90 minutos

```
CREATE TABLE productos (  
    producto_id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre      VARCHAR(100) NOT NULL,  
    categoria   VARCHAR(60),  
    precio      DECIMAL(10,2) NOT NULL CHECK (precio > 0),  
    stock       INT DEFAULT 0  
);  
  
CREATE TABLE pedidos (  
    pedido_id   INT AUTO_INCREMENT PRIMARY KEY,  
    cliente_id  INT NOT NULL,  
    producto_id INT NOT NULL,  
    cantidad    INT NOT NULL CHECK (cantidad > 0),  
    fecha_pedido DATE DEFAULT (CURRENT_DATE),  
    estado      VARCHAR(20) DEFAULT "pendiente"  
    CHECK (estado IN ("pendiente","entregado","cancelado")),  
    FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id),  
    FOREIGN KEY (producto_id) REFERENCES productos(producto_id)  
);  
  
-- =====  
-- DML: DATOS DE PRUEBA  
-- =====  
INSERT INTO clientes VALUES  
    (1,"Ana Lopez","ana@mail.com","Bogota","2023-01-15"),  
    (2,"Carlos Ruiz","carlos@mail.com","Medellin","2023-03-22"),  
    (3,"Maria Torres","maria@mail.com","Cali","2023-05-10"),  
    (4,"Pedro Gomez","pedro@mail.com","Bogota","2023-07-08"),  
    (5,"Sofia Herrera","sofia@mail.com","Barranquilla","2023-09-01"),  
    (6,"Luis Martinez","luis@mail.com","Bogota","2024-01-20"),  
    (7,"Camila Vargas","camila@mail.com","Cali","2024-02-14"),  
    (8,"Diego Morales","diego@mail.com","Medellin","2024-03-30");  
  
INSERT INTO productos VALUES  
    (1,"Laptop Pro 15","Computadores",3500000.00,12),  
    (2,"Mouse Inalambrico","Perifericos",85000.00,50),  
    (3,"Teclado Mecanico","Perifericos",220000.00,30),  
    (4,"Monitor 27","Pantallas",1200000.00,8),  
    (5,"Auriculares BT","Audio",350000.00,25),  
    (6,"Webcam HD","Perifericos",180000.00,20),  
    (7,"Disco SSD 1TB","Almacenamiento",420000.00,40),  
    (8,"Tablet 10","Moviles",1800000.00,6);  
  
INSERT INTO pedidos VALUES  
    (1,1,1,1,"2024-01-10","entregado"), (2,1,2,2,"2024-01-15","entregado"),  
    (3,2,3,1,"2024-02-05","entregado"), (4,2,5,1,"2024-02-20","cancelado"),  
    (5,3,4,1,"2024-03-01","entregado"), (6,3,7,2,"2024-03-15","pendiente"),  
    (7,4,2,3,"2024-04-02","entregado"), (8,4,6,1,"2024-04-10","pendiente"),  
    (9,5,8,1,"2024-04-18","entregado"), (10,6,1,2,"2024-05-05","entregado"),  
    (11,6,3,1,"2024-05-12","pendiente"), (12,7,5,2,"2024-05-20","entregado"),  
    (13,1,7,1,"2024-06-01","entregado"), (14,8,4,1,"2024-06-10","cancelado"),  
    (15,5,2,4,"2024-06-15","entregado"), (16,3,1,1,"2024-07-01","pendiente");
```

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 1 Duración: 90 minutos

PREGUNTAS — PARCIAL No.1

Cada pregunta combina múltiples temas. Todas tienen el mismo nivel de complejidad.

No.	Temas	Enunciado de la Pregunta
1	DDL DML	Agregue a la tabla pedidos una columna total_valor DECIMAL(12,2) generada automáticamente como la multiplicación de cantidad por el precio del producto (columna calculada persistida con AS ... STORED, o en su defecto agréguela como columna normal y luego actualice su valor mediante un UPDATE con JOIN entre pedidos y productos). Finalmente, agregue un índice sobre la columna estado. Clausulas requeridas: ALTER TABLE, UPDATE ... JOIN, CREATE INDEX
2	DDL Vista	Cree la tabla log_cambios_estado (log_id PK AI, pedido_id FK, estado_anterior VARCHAR(20), estado_nuevo VARCHAR(20), fecha_cambio DATETIME DEFAULT NOW()). A continuación, cree una vista llamada vista_log_reciente que muestre los últimos 10 registros de log_cambios_estado ordenados por fecha_cambio descendente. Clausula requeridas: CREATE TABLE, FOREIGN KEY, CREATE VIEW, ORDER BY, LIMIT
3	DML Multi	Realice las siguientes operaciones en una misma sesión: (a) Inserte un nuevo cliente (nombre=Laura Rios, email=laura@mail.com, ciudad=Manizales). (b) Inserte un pedido para ese cliente del producto_id=3 con cantidad=2 y estado=pendiente. (c) Actualice el stock del producto_id=3 decrementándolo en 2. (d) Consulte con un JOIN el nombre del cliente, nombre del producto y estado del pedido recién creado. Clausula requeridas: INSERT, UPDATE, SELECT con JOIN, WHERE
4	DML Sub	Actualice el precio de todos los productos cuyo stock sea menor al promedio de stock de su misma categoría (use subconsulta correlacionada), incrementando el precio un 8%. Luego elimine los pedidos con estado cancelado cuyos clientes no tengan ningún otro pedido en estado entregado (use subconsulta con NOT EXISTS). Clausulas requeridas: UPDATE con subconsulta correlacionada, DELETE con NOT EXISTS
5	Multi Sub	Liste el nombre del cliente, ciudad, nombre del producto, cantidad y fecha_pedido de todos los pedidos entregados cuyo total (cantidad * precio) supere el promedio general de totales de pedidos entregados. Ordene los resultados por total descendente. Clausulas requeridas: JOIN tres tablas, WHERE con subconsulta escalar AVG, ORDER BY DESC
6	Multi Vista	Cree la vista vista_ventas_ciudad que muestre: ciudad, total_pedidos_entregados, suma_ingresos (SUM de cantidad*precio) y promedio_ingreso_por_pedido. Luego consulte la vista para mostrar solo las ciudades cuyo suma_ingresos supere los 5,000,000, ordenadas de mayor a menor. Clausula requeridas: CREATE VIEW con JOIN, GROUP BY, CREATE INDEX opcional, SELECT FROM vista con WHERE y ORDER BY
7	Sub Vista	Cree la vista vista_productos_populares que liste los productos que hayan sido pedidos por más de un cliente distinto (en pedidos entregados). La vista debe mostrar: producto_id, nombre, categoría, precio y total_clientes_distintos. Luego use la vista para obtener únicamente los productos de la categoría Perifericos. Clausula requeridas: CREATE VIEW con subconsulta o HAVING COUNT(DISTINCT), SELECT FROM vista con WHERE
8	Func Multi	Cree la función fn_ingreso_cliente(p_cliente_id INT) que retorne el ingreso total acumulado de un cliente (suma de cantidad*precio solo para pedidos entregados,

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 1 Duración: 90 minutos

		usando JOIN entre pedidos y productos). Luego use esa función en un SELECT sobre la tabla clientes para mostrar nombre, ciudad y su ingreso_total, ordenados de mayor a menor ingreso. Clausulas requeridas: CREATE FUNCTION con SELECT JOIN, RETURN; SELECT usando la función en la lista de columnas
9	Func Sub	Cree la función fn_stock_suficiente(p_producto_id INT, p_cantidad_solicitada INT) que retorne 1 si el stock actual del producto es mayor o igual a la cantidad solicitada, o 0 en caso contrario. Luego escriba una consulta que liste nombre y stock de todos los productos donde fn_stock_suficiente(producto_id, 5) = 0, es decir, productos con menos de 5 unidades disponibles. Clausulas requeridas: CREATE FUNCTION, SELECT con WHERE usando la función, subconsulta o logica equivalente
10	Proc DML	Cree el procedimiento sp_actualizar_estado_pedido(p_pedido_id INT, p_nuevo_estado VARCHAR(20)) que: (a) Verifique que el pedido exista (si no, retorne mensaje de error). (b) Inserte un registro en log_cambios_estado con el estado anterior y el nuevo. (c) Actualice el estado del pedido. (d) Si el nuevo estado es cancelado, restaure el stock del producto correspondiente. Clausula requeridas: CREATE PROCEDURE, SELECT INTO var, IF/ELSE, INSERT, UPDATE dos tablas
11	Proc Multi	Cree el procedimiento sp_resumen_cliente(p_cliente_id INT) que ejecute y retorne en un solo SELECT: nombre del cliente, ciudad, total de pedidos por estado (use SUM con CASE WHEN para contar pedidos entregados, pendientes y cancelados en columnas separadas) y el ingreso total solo de pedidos entregados. Clausula requeridas: CREATE PROCEDURE, SELECT con JOIN, SUM(CASE WHEN ...), GROUP BY
12	Proc Vista	Cree la vista vista_pedidos_pendientes que muestre pedido_id, nombre del cliente, nombre del producto, cantidad, precio unitario y dias_espera (DATEDIFF entre CURDATE() y fecha_pedido) para todos los pedidos con estado pendiente. Luego cree el procedimiento sp_alertar_retrasos(p_dias_limite INT) que consulte esa vista y retorne los pedidos cuyo dias_espera supere p_dias_limite. Clausula requeridas: CREATE VIEW con JOIN y DATEDIFF, CREATE PROCEDURE con SELECT FROM vista con WHERE
13	DDL Func Sub	Agregue la columna descuento DECIMAL(5,2) DEFAULT 0 a la tabla productos con una restricción CHECK que garantice valores entre 0 y 50. Cree la función fn_precio_final(p_producto_id INT) que retorne el precio del producto aplicando su descuento (precio * (1 - descuento/100)). Luego escriba una consulta que muestre nombre, precio, descuento y precio_final para los 3 productos con mayor precio_final, usando la función. Clausulas requeridas: ALTER TABLE ADD COLUMN CHECK, CREATE FUNCTION con SELECT, SELECT ORDER BY DESC LIMIT 3
14	DML Proc Multi	Cree el procedimiento sp_registrar_pedido(p_cliente_id INT, p_producto_id INT, p_cantidad INT) que: (a) Valide que el cliente exista. (b) Valide que el stock sea suficiente. (c) Inserte el pedido con estado pendiente. (d) Actualice el stock descontando la cantidad. (e) Retorne con un SELECT JOIN el pedido recién creado con nombre del cliente y nombre del producto. Clausulas requeridas: CREATE PROCEDURE, SELECT INTO var para validar, IF, INSERT, UPDATE, SELECT con JOIN al final
15	Multi Func Vista	Cree la funcion fn_clasificar_producto(p_producto_id INT) que retorne: PREMIUM si el precio > 1,000,000; ESTANDAR si esta entre 200,000 y 1,000,000; BASICO si es menor a 200,000. Luego cree la vista vista_catalogo_clasificado que muestre nombre, categoria, precio, clasificacion (usando la funcion) y stock para todos los

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 1 Duración: 90 minutos

		productos. Finalmente, consulte la vista mostrando solo los productos PREMIUM con stock > 5. Clausulas requeridas: CREATE FUNCTION con CASE, CREATE VIEW usando la función, SELECT FROM vista con WHERE compuesto
16	Sub Multi Vista	Cree la vista vista_clientes_vip que contenga el cliente_id, nombre, ciudad y total_pedidos_entregados de clientes que hayan realizado mas pedidos entregados que el promedio de pedidos entregados por cliente (use subconsulta en el HAVING). Luego escriba una consulta sobre esa vista junto con un JOIN a pedidos y productos para listar el detalle de los últimos 2 pedidos de cada cliente VIP, mostrando nombre del cliente, nombre del producto y fecha_pedido. Clausulas requeridas: CREATE VIEW con HAVING > subconsulta AVG, SELECT con JOIN y subconsulta o RANK/ROW_NUMBER